

Learn Mate - AI assistant for smarter learning and study support.

Project title: AI STUDY-BUDDY

Team ID: SWTID-2026-3742

Team Size: 4

Team Leader:

KEERTHANA N (C24UG185CSC020)

Team member:

KOUSHIKA K(C24UG185CSC021)

LAVANYA B(C24UG185CSC022)

MAHALAKSHMI M(C24UG185CSC023)

EMAIL ID:

Keerthananallamuthu@gmail.com

kkoushi05012007@gmail.com

llavanya5738@gmail.com

mahasakthi430@gmail.com

DESCRIPTION

AI StudyBuddy is an AI-powered learning assistance platform developed to simplify the study process for students by leveraging modern web technologies and Generative AI. The application provides an intelligent backend system that enables students to upload study materials, generate concise summaries, create flashcards, produce quizzes, and receive personalized study plans through AI-generated responses. The backend is developed using Node.js and Express.js, following a RESTful API architecture that ensures scalability, maintainability, and secure communication between the frontend and backend services. MongoDB serves as the primary NoSQL database, while Mongoose ODM provides schema validation, data modeling, and efficient database interactions. To secure user information and

protected resources, the application implements JWT (JSON Web Token) based authentication along with Role-Based Access Control (RBAC).

Users are authenticated before accessing AI-powered features, ensuring that only authorized users can utilize premium learning services. Passwords are securely encrypted using bcryptjs, preventing unauthorized access to user credentials.

One of the key highlights of AI StudyBuddy is its integration with the Google Gemini AI API, which enables the application to perform advanced natural language processing tasks. Instead of relying on predefined templates, the backend dynamically communicates with the Gemini model to generate high-quality educational content based on the study material provided by the user.

Scenario-Based Case Study

Maha is a second-year CS student preparing for multiple semester examinations while simultaneously learning new technical skills for internships and placement opportunities. His study materials are scattered across lecture notes, PDFs, textbooks, online articles, and handwritten notes, making it difficult to organize and revise effectively. Due to the large volume of content and limited preparation time, Rahul often struggles to identify important concepts, create revision notes, and evaluate his understanding before examinations.

Traditional study methods require significant manual effort to summarize lengthy materials, prepare practice questions, and create revision schedules. This process is repetitive, time-consuming, and often results in inconsistent learning outcomes.

Problem

Rahul encounters several challenges during his preparation:

- Difficulty understanding lengthy study materials within limited time.
- Manual preparation of notes consumes a significant amount of study time.
- Creating flashcards and revision questions is repetitive and labor-intensive.
- Lack of personalized study plans based on available time and examination schedules.
- No centralized platform to securely store AI-generated learning resources.
- Difficulty assessing knowledge through self-generated quizzes.
- Switching between multiple applications for note-taking, quizzes, and scheduling reduces productivity.

SOLUTION

The **AI StudyBuddy** backend provides a centralized, AI-powered learning platform that automates several academic tasks through intelligent content generation. The system enables authenticated users to upload or enter study materials and leverage the **Google Gemini AI API** to generate educational resources automatically. The backend processes incoming requests through secure RESTful APIs and performs.

Usage

Student Usage

Students register and log in to the application before accessing AI-powered learning features. After authentication, they can upload study material or provide textual content and choose the desired AI functionality, such as generating summaries, quizzes, flashcards, or personalized study plans. The generated content can be reviewed immediately and stored for future reference.

Administrator Usage

Administrators manage registered users, monitor application activity, oversee system usage, and maintain platform integrity. Administrative privileges include managing user accounts, monitoring AI service utilization, reviewing system logs, and ensuring smooth backend operations.

Outcome

AI StudyBuddy uses AI to make learning easier and faster for students. It automatically creates study materials, reducing the time needed for manual preparation. JWT authentication and MongoDB help keep student data secure and organized. Google Gemini AI provides intelligent and personalized learning support. Overall, it helps students revise effectively and prepare better for exams.

Software Requirements

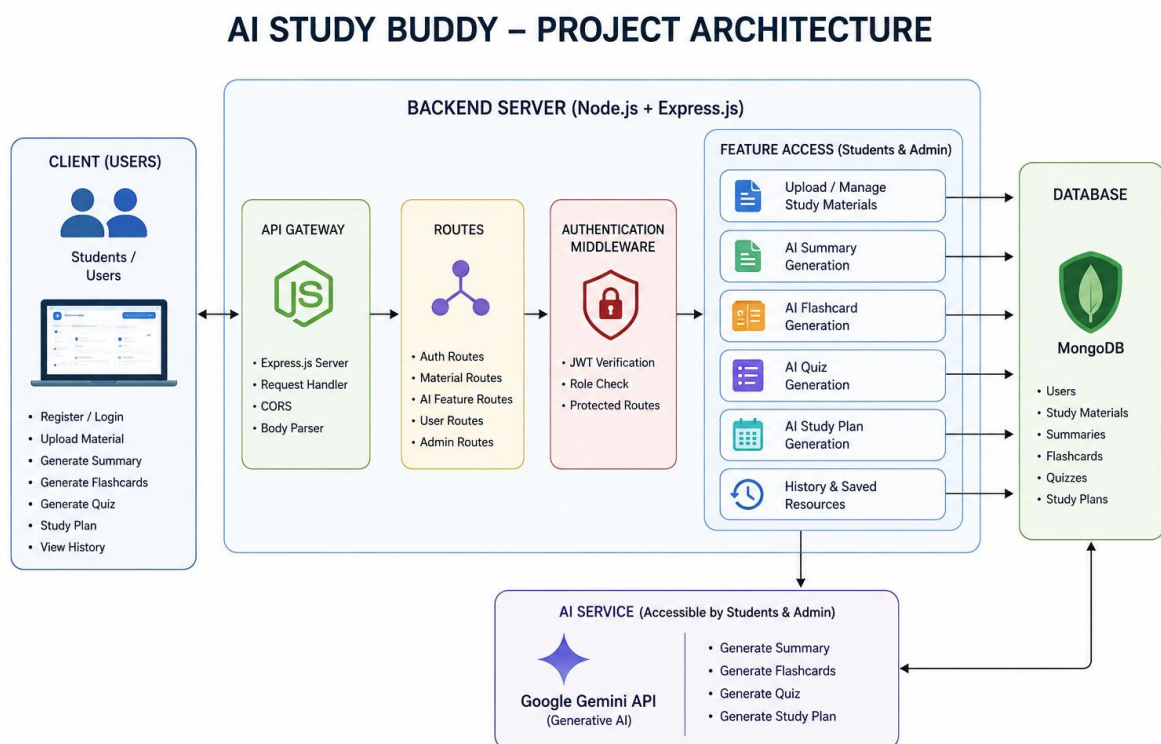
- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Express.js:** Lightweight routing web framework to construct backend RESTful entry points.

- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

Epic 1: Project Architecture

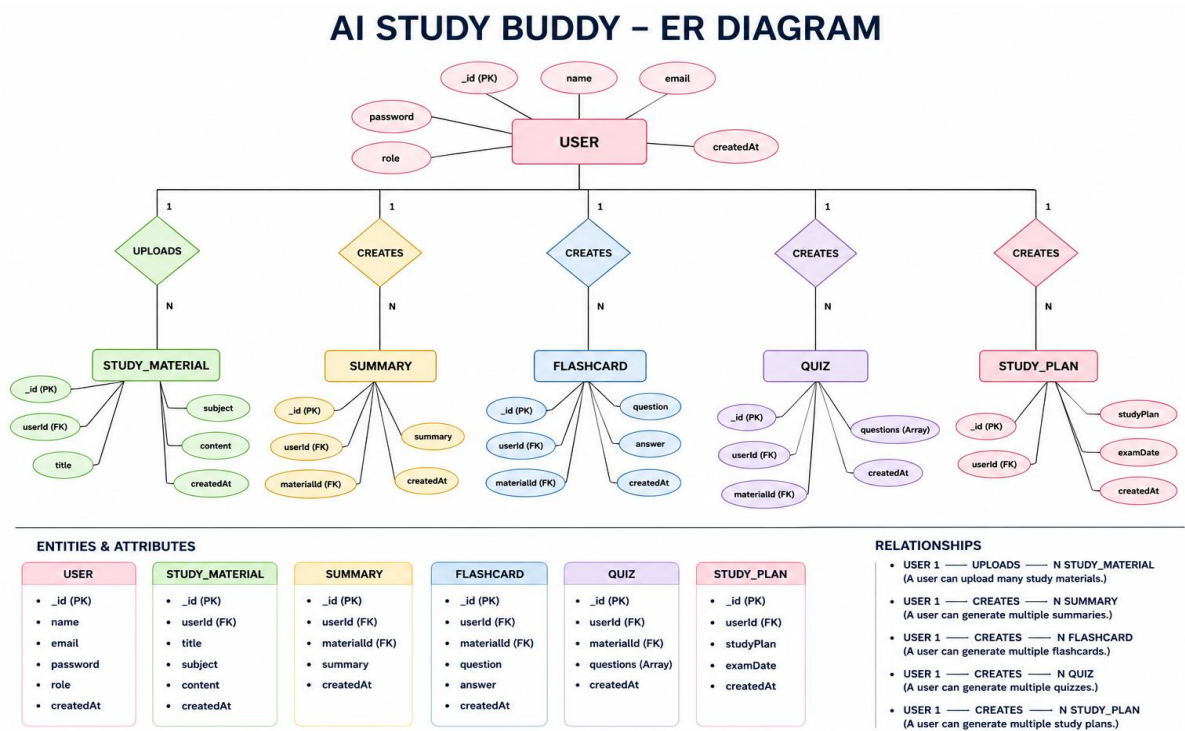


The **AI StudyBuddy** backend follows a modular RESTful architecture designed to provide secure, scalable, and AI-powered learning services. The system processes client requests through Express.js, validates users using JWT authentication, interacts with the MongoDB database for data storage, and communicates with the Google Gemini AI API to generate intelligent study resources.

Components Definition

- **Client Application:** The frontend (React.js) allows users to register, log in, upload study material, and access AI-powered features such as summaries, quizzes, flashcards, and study plans through REST APIs.
 - **Express Server:** The Express.js server receives HTTP requests, handles routing, processes middleware, and sends appropriate responses to the client.
 - **Authentication Middleware:** JWT-based middleware verifies user identity, protects private routes, and ensures that only authenticated users can access AI services and study resources.
- Route Handlers:** Routes manage API endpoints related to user authentication, study materials, AI features, and administrative operations.
- **AI Service Layer:** A dedicated AI service communicates with the **Google Gemini API** to generate summaries, quizzes, flashcards, and personalized study plans based on the user's study content.
 - **Database Layer:** MongoDB, together with Mongoose ODM, stores user information, uploaded study materials, and AI-generated learning resources securely.

Entity-Relationship (ER) Diagram Description



The AI StudyBuddy backend manages data through interconnected MongoDB collections that store user information, study materials, and AI-generated learning resources. Each entity is linked using logical relationships to ensure efficient data management and retrieval.

1. User Entity

Stores the details of registered users and administrators.

Attributes:

- `_id` (Primary Key - ObjectId)
- `name` (String, Required)
- `email` (String, Required, Unique)
- `password` (String, Hashed)
- `role` (String - Student/Admin)
- `createdAt`

2. Study Material Entity

Stores study content uploaded by students for AI processing.

Attributes:

- `_id` (Primary Key)
- `userId` (Foreign Key → User)
- `title`
- `subject`
- `content`
- `createdAt`

3. Summary Entity

Stores AI-generated summaries.

Attributes:

- `_id` (Primary Key)

- userId (Foreign Key → User)
- materialId (Foreign Key → Study Material)
- summary
- createdAt

4. Flashcard Entity

Stores AI-generated flashcards.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User)
- materialId (Foreign Key → Study Material)
- question
- answer
- createdAt

5. Quiz Entity

Stores AI-generated quizzes.

Attributes:

- _id (Primary Key)
- userId (Foreign Key → User)
- materialId (Foreign Key → Study Material)
 - questions
 - createdAt

6. Study Plan Entity

Stores personalized study plans generated by AI.

Attributes:

- `_id` (Primary Key)
- `userId` (Foreign Key → User)
- `studyPlan`
- `examDate`
- `createdAt`

Key Relationships

- **User → Study Material (One-to-Many):** A user can upload multiple study materials.
- **Study Material → Summary (One-to-One):** Each uploaded material can have one AI-generated summary.
- **Study Material → Flashcards (One-to-Many):** One study material can generate multiple flashcards.
- **Study Material → Quiz (One-to-One):** One study material can generate one quiz containing multiple questions.
- **User → Study Plan (One-to-Many):** A user can generate multiple study plans based on different subjects or examination schedules.
- **User → AI Resources (One-to-Many):** All AI-generated summaries, flashcards, quizzes, and study plans are associated with the user who generated them.

Key Features

The AI StudyBuddy backend provides intelligent learning assistance through secure and scalable REST APIs. The major features of the system are as follows:

- **Secure User Authentication:** Implements JWT-based authentication with encrypted passwords using bcrypt for secure user access.
- **Study Material Management:** Allows users to upload and manage study materials that serve as input for AI-powered learning features.
- **AI Summary Generation:** Generates concise and easy-to-understand summaries from uploaded study content using the Google Gemini API.

- **AI Flashcard Generation:** Automatically creates flashcards from study materials to improve concept revision and active recall.
- **AI Quiz Generation:** Produces multiple-choice quizzes based on uploaded content, enabling students to test their knowledge.

Roles and Responsibilities

Student

- Register and log in securely.
- Upload study materials.
- Generate summaries and flashcards.
- Create AI-based quizzes.
- Get personalized study plans.
- View and save learning resources.

Administrator

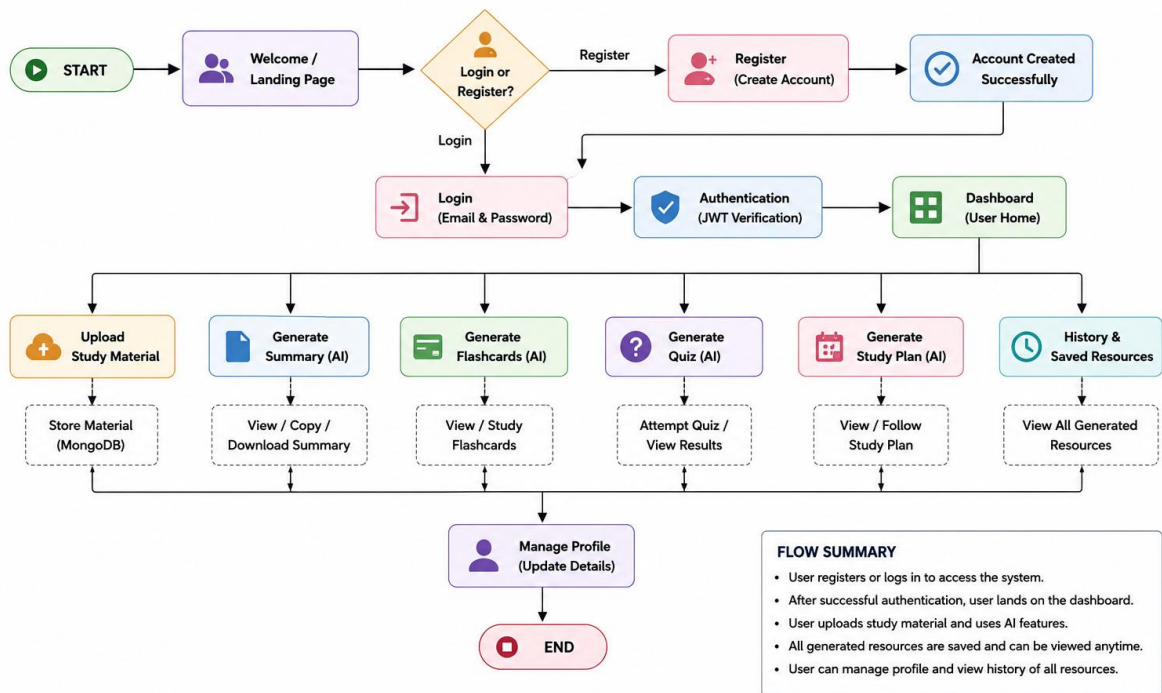
- Manage registered users.
- Monitor the application.
- Manage user accounts.
- Maintain system security.
- Check the proper working of AI services.

User Flow

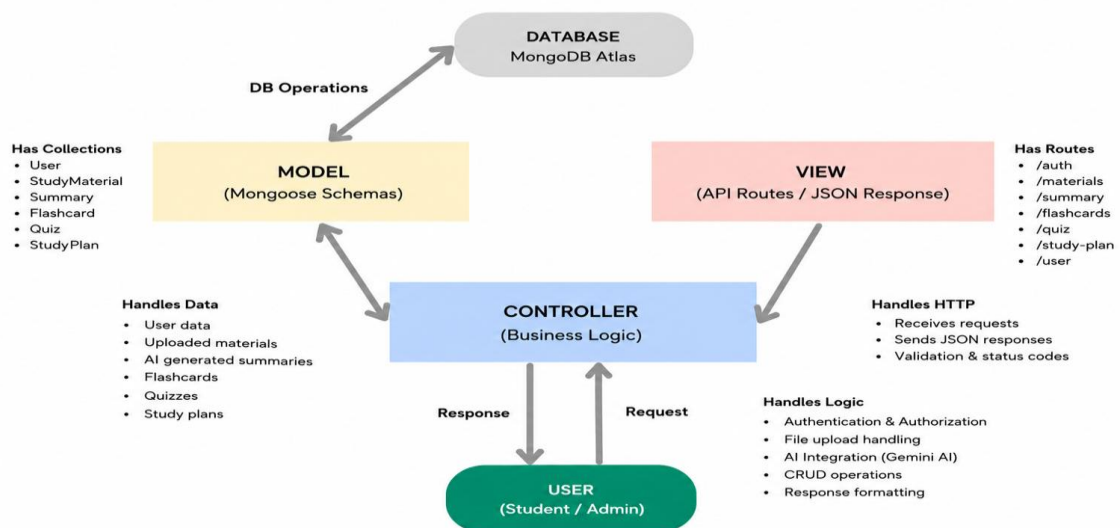
- User **registers or logs in**.
- User enters the **dashboard**.
- User **uploads study materials**.
- User selects an **AI feature**.
- Gemini AI processes the content.
- System generates **summaries, flashcards, quizzes, or study plans**.

- User can **view and save** the generated resources.

— USER FLOW DIAGRAM – AI STUDY BUDDY —



MVC Architecture Pattern



The AI StudyBuddy backend follows the **Model-View-Controller (MVC)** architecture to organize the application into separate layers, making the code modular, maintainable, and scalable.

Model Layer (Data Layer)

The Model layer defines the MongoDB collections using Mongoose schemas. It is responsible for managing user information, uploaded study materials, and AI-generated learning resources while ensuring data validation and database interactions.

View Layer (API Response Layer)

Since AI StudyBuddy is a backend REST API, there is no traditional user interface in the View layer. Instead, the API returns structured JSON responses to the React frontend, allowing users to interact with the application through REST endpoints.

Controller Layer (Business Logic)

The Controller layer processes incoming API requests, validates user input, performs authentication and authorization, communicates with the database models, invokes the Google Gemini AI service when required, and returns the appropriate response to the client.

MVC Benefits

The MVC architecture separates data management, business logic, and client communication into independent components. This improves code readability, simplifies debugging and testing, and allows new features to be integrated without affecting existing modules.

Epic2: Project Setup and Configuration

Creating the Project Folder

1. Create a new folder named AI StudyBuddy .
2. Open the project folder in Visual Studio Code.
3. Initialize the project using Node.js and install the required dependencies.

Project Initialization

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer  
@google/genai
```

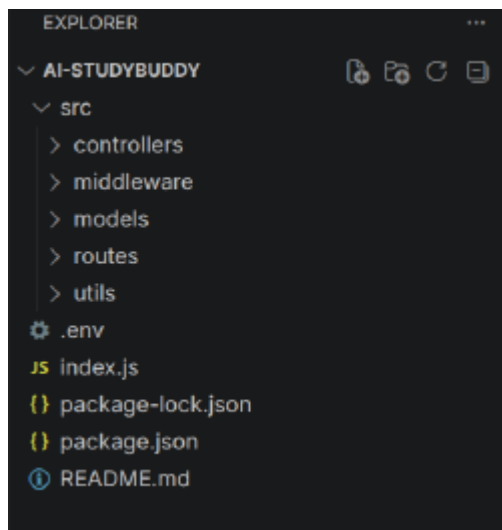
Install Nodemon for development

```
npm install --save-dev nodemon
```

Create the Following Files

- index.js
- .env
- package.json
- .gitignore

Create the Following Folder Structure



Environment Configuration

Create a **.env** file in the project root and configure the required environment variables.

PORT=5000

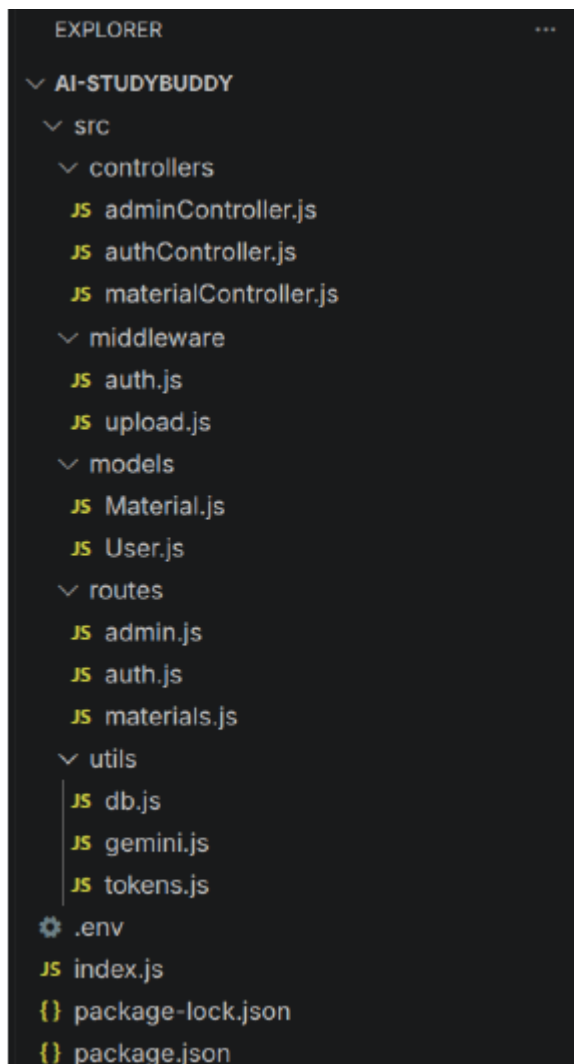
MONGO_URI=your_mongodb_connection_string

JWT_SECRET=your_jwt_secret

GEMINI_API_KEY=your_gemini_api_key

Backend Development

The AI StudyBuddy backend is developed using **Node.js**, **Express.js**, and **MongoDB**, following a modular architecture.



Core System Script Implementations

1. Application Entry Point (index.js)
2. Database Connection (src/utils/db.js)
3. Authentication Middleware (src/middleware/auth.js)
4. AI Integration (src/utils/gemini.js)
5. File Upload Middleware (src/middleware/upload.js)

CODING

```
JS index.js > ...
1  require("dotenv").config();
2  require("express-async-errors");
3
4  const express = require("express");
5  const cors = require("cors");
6  const connectDB = require("../src/Utils/db");
7
8  const app = express();
9
10 // Middleware
11 app.use(express.json());
12 app.use(cors({ origin: process.env.CLIENT_URL || "http://localhost:3000" }));
13
14 // Routes
15 app.use("/api/auth", require("../src/routes/auth"));
16 app.use("/api/materials", require("../src/routes/materials"));
17 app.use("/api/admin", require("../src/routes/admin"));
18
19 // Health check
20 app.get("/", (req, res) => res.json({ message: "AI StudyBuddy API is running" }));
21
22 // Global error handler
23 app.use((err, req, res, next) => {
24   console.error(err.message);
25   const status = err.status || 500;
26   res.status(status).json({ message: err.message || "Something went wrong" });
27 });
28
29 const PORT = process.env.PORT || 5000;
30
31 connectDB().then(() => {
32   app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
33 });
34
```

```
src > utils > JS db.js > ...
1  const mongoose = require("mongoose");
2  const dotenv = require("dotenv");
3
4  dotenv.config();
5
6  const connectDB = async () => {
7    const conn = await mongoose.connect(process.env.MONGO_URI);
8    console.log(`MongoDB connected: ${conn.connection.host}`);
9  };
10
11 module.exports = connectDB;
12
```

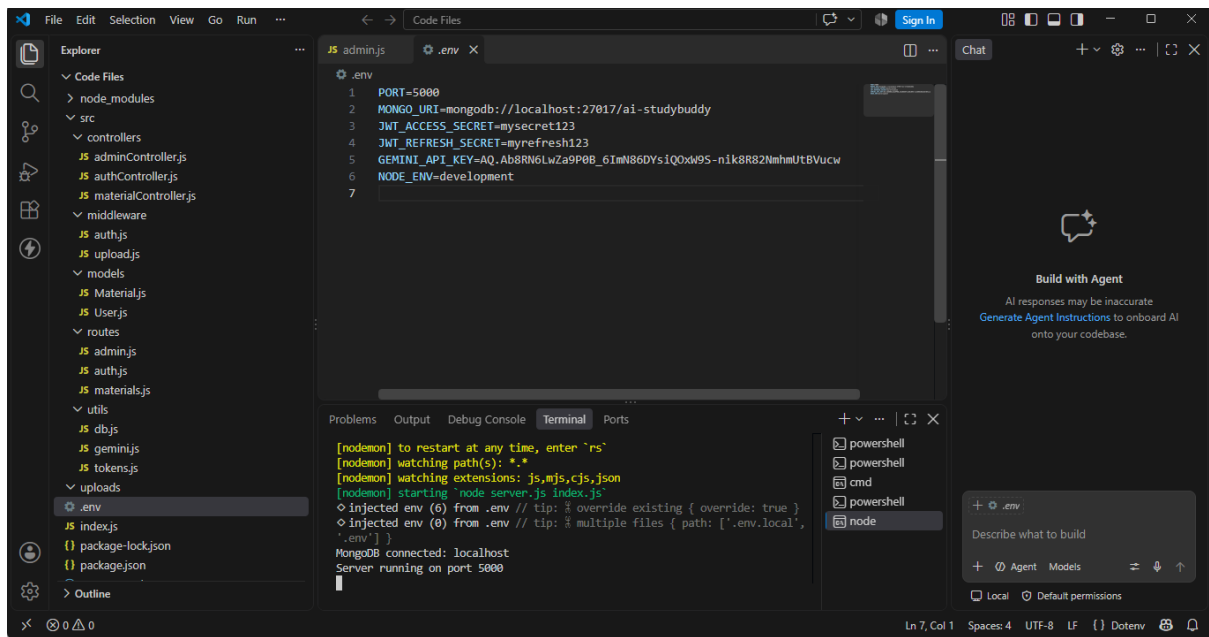
src > middleware > **JS** auth.js > ...

```
1  const jwt = require("jsonwebtoken");
2
3  const protect = (req, res, next) => {
4    const authHeader = req.headers.authorization;
5    if (!authHeader || !authHeader.startsWith("Bearer ")) {
6      return res.status(401).json({ message: "Not authenticated" });
7    }
8
9    const token = authHeader.split(" ")[1];
10
11    try {
12      const decoded = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
13      req.user = decoded;
14      next();
15    } catch {
16      return res.status(401).json({ message: "Invalid or expired token" });
17    }
18  };
19
20  const adminOnly = (req, res, next) => {
21    if (req.user.role !== "admin") {
22      return res.status(403).json({ message: "Admins only" });
23    }
24    next();
25  };
26
27  module.exports = { protect, adminOnly };
```

src > middleware > **JS** upload.js > ...

```
1  const multer = require("multer");
2  const path = require("path");
3
4  const storage = multer.diskStorage({
5    destination: (req, file, cb) => cb(null, "uploads/"),
6    filename: (req, file, cb) => {
7      const unique = Date.now() + "-" + Math.round(Math.random() * 1e9);
8      cb(null, unique + path.extname(file.originalname));
9    },
10  });
11
12  const fileFilter = (req, file, cb) => {
13    const allowed = [".txt", ".md", ".pdf"];
14    const ext = path.extname(file.originalname).toLowerCase();
15    if (allowed.includes(ext)) {
16      cb(null, true);
17    } else {
18      cb(new Error("Only .txt, .md, and .pdf files are allowed"));
19    }
20  };
21
22  const upload = multer({ storage, fileFilter, limits: { fileSize: 5 * 1024 * 1024 } });
23
24  module.exports = upload;
```

COMPILE



OUTPUT

